

CSE 3204
Formal Language, Automata and Computability



Lecture 1
Deterministic Finite Automata

Course Information

- Course Teacher: Dr. Mosarrat Jahan
Associate Professor, Dept. of CSE, DU
- Contact Email: mosarratjahan@cse.du.ac.bd
- Text Book: Introduction to Automata Theory Languages, and Computation (3rd Edition)
-- John E. Hopcroft, Rajeev Motwani and Jeffrey D. Ullman
- Classroom code: z6nssbog

Evaluation Policy

- Incourse exam— 20%
- Assignment—5%
- Attendance—5%

Finite Automaton (FA)

- Informally, a state diagram that comprehensively captures all possible states and transitions that a machine can take while responding to a stream or sequence of input symbols
 - Deterministic Finite Automata (DFA)
 - Non-deterministic Finite Automata (NFA)

Deterministic Finite Automata (DFA)

- A Deterministic Finite Automaton (DFA) consists of:
 - $Q \implies$ a finite set of states
 - $\Sigma \implies$ a finite set of input symbols (alphabet)
 - $q_0 \implies$ a start state
 - $F \implies$ set of accepting states
 - $\delta \implies$ a transition function, which is a mapping between $Q \times \Sigma \implies Q$
- A DFA is defined by the 5-tuple:
 - $(Q, \Sigma, q_0, F, \delta)$
- A language of a DFA is a set of all strings consists of symbols from Σ accepted by DFA

How DFA processes Strings

- Input: a word w consists of symbols in Σ
- Question: Is w acceptable by the DFA?
- Steps:
 - Start at the “start state” q_0
 - For every input symbol in the sequence w do
 - Compute the next state from the current state, given the current input symbol in w and the transition function
 - If after all symbols in w are consumed, the current state is one of the accepting states (F) then *accept* w ;
 - Otherwise, *reject* w .

Example: A DFA Processing Strings

- A DFA that accepts all strings of 0's and 1's having the sequence 01 somewhere in the string.
- $L = \{w \mid w \text{ is of form } x01y \text{ for some strings } x \text{ and } y \text{ consisting of 0's and 1's only}\}$
- Equivalent:
$$L = \{x01y \mid x \text{ and } y \text{ are any strings of 0's and 1's}\}$$

Example: A DFA Processing Strings

- Three considerations:

1. Has it already seen 01? If so, then it accepts every sequence of further inputs; i.e., it will only be in accepting states from now on.
2. Has it never seen 01, but its most recent input was 0, so if it now sees a 1, it will have seen 01 and can accept everything it sees from here on?
3. Has it never seen 01, but its last input was either nonexistent (it just started) or it last saw a 1? In this case, A cannot accept until it first sees a 0 and then sees a 1 immediately after.

For condition 3: $\delta(q_0, 1) = q_0$, $\delta(q_0, 0) = q_2$

For condition 2: $\delta(q_2, 1) = q_1$, $\delta(q_2, 0) = q_2$

For condition 3: $\delta(q_1, 1) = q_1$, $\delta(q_1, 0) = q_1$

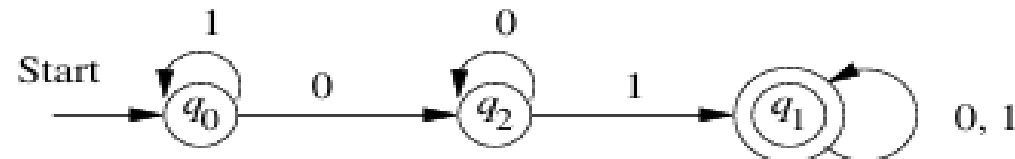
$A = (\{q_0, q_1, q_2\}, \{0,1\}, \delta, q_0, \{q_1\})$

Notation of DFA: Transition Diagram

A *transition diagram* for a DFA $A = (Q, \Sigma, \delta, q_0, F)$ is a graph defined as follows:

- a) For each state in Q there is a node.
- b) For each state q in Q and each input symbol a in Σ , let $\delta(q, a) = p$. Then the transition diagram has an arc from node q to node p , labeled a . If there are several input symbols that cause transitions from q to p , then the transition diagram can have one arc, labeled by the list of these symbols.
- c) There is an arrow into the start state q_0 , labeled *Start*. This arrow does not originate at any node.
- d) Nodes corresponding to accepting states (those in F) are marked by a double circle. States not in F have a single circle.

Example: Transition diagram of a DFA accepting all strings containing 01 as substring.



Notation of DFA: Transition Tables

- A conventional tabular representation of δ .
- Rows correspondence to state and columns represents symbol.
- Start state is marked with an arrow and the final states are marked with a *.
- Example: Transition table of a DFA accepting all strings containing 01.

	0	1
$\rightarrow q_0$	q_2	q_0
$*q_1$	q_1	q_1
q_2	q_2	q_1

Extending Transition Function to String

- An extended transition function $\hat{\delta}$ takes a state q and an input string w and returns a state p denoted by $\hat{\delta}(q, w) = p$.
that is, starting from state q after processing all symbols of w , DFA reaches p .
- $\hat{\delta}$ can be defined by induction on the length of input string.

BASIS: $\hat{\delta}(q, \epsilon) = q$. That is, if we are in state q and read no inputs, then we are still in state q .

INDUCTION: Suppose w is a string of the form xa ; that is, a is the last symbol of w , and x is the string consisting of all but the last symbol.³ For example, $w = 1101$ is broken into $x = 110$ and $a = 1$. Then

$$\hat{\delta}(q, w) = \delta(\hat{\delta}(q, x), a) \quad (2.1)$$

Now (2.1) may seem like a lot to take in, but the idea is simple. To compute $\hat{\delta}(q, w)$, first compute $\hat{\delta}(q, x)$, the state that the automaton is in after processing all but the last symbol of w . Suppose this state is p ; that is, $\hat{\delta}(q, x) = p$. Then $\hat{\delta}(q, w)$ is what we get by making a transition from state p on input a , the last symbol of w . That is, $\hat{\delta}(q, w) = \delta(p, a)$.

Example: Extending Transition Function to String

- Design a DFA accepting language L

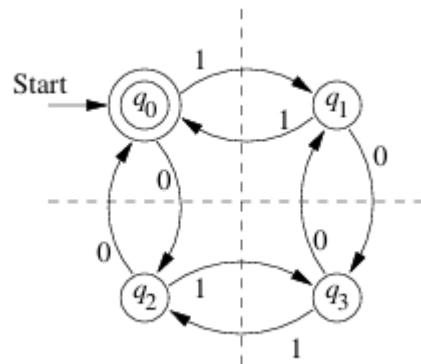
$$L = \{w \mid w \text{ has both an even number of 0's and an even number of 1's}\}$$

q_0 : Both the number of 0's seen so far and the number of 1's seen so far are even.

q_1 : The number of 0's seen so far is even, but the number of 1's seen so far is odd.

q_2 : The number of 1's seen so far is even, but the number of 0's seen so far is odd.

q_3 : Both the number of 0's seen so far and the number of 1's seen so far are odd.



Transition diagram for the DFA

	0	1
* $\rightarrow q_0$	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

Transition table for the DFA

Example: Extending Transition Function to String

- We need to show $\hat{\delta}(q_0, 110101) = q_0$.
- $\hat{\delta}(q_0, \epsilon) = q_0$.
- $\hat{\delta}(q_0, 1) = \delta(\hat{\delta}(q_0, \epsilon), 1) = \delta(q_0, 1) = q_1$.
- $\hat{\delta}(q_0, 11) = \delta(\hat{\delta}(q_0, 1), 1) = \delta(q_1, 1) = q_0$.
- $\hat{\delta}(q_0, 110) = \delta(\hat{\delta}(q_0, 11), 0) = \delta(q_0, 0) = q_2$.
- $\hat{\delta}(q_0, 1101) = \delta(\hat{\delta}(q_0, 110), 1) = \delta(q_2, 1) = q_3$.
- $\hat{\delta}(q_0, 11010) = \delta(\hat{\delta}(q_0, 1101), 0) = \delta(q_3, 0) = q_1$.
- $\hat{\delta}(q_0, 110101) = \delta(\hat{\delta}(q_0, 11010), 1) = \delta(q_1, 1) = q_0$.

Language of a DFA

Now, we can define the *language* of a DFA $A = (Q, \Sigma, \delta, q_0, F)$. This language is denoted $L(A)$, and is defined by

$$L(A) = \{w \mid \hat{\delta}(q_0, w) \text{ is in } F\}$$

That is, the language of A is the set of strings w that take the start state q_0 to one of the accepting states. If L is $L(A)$ for some DFA A , then we say L is a *regular language*.

- Section 2.2 of Chapter 2.